

Materi Kecerdasan Buatan & Rekayasa Sistem

Bab 1: Pendahuluan Rekayasa Sistem Modern

Dalam pengembangan perangkat lunak modern, khususnya rekayasa sistem kecerdasan buatan, modularitas menjadi fondasi utama yang menentukan keandalan jangka panjang. Desain komponen modular bekerja dengan mengisolasi kompleksitas sistem ke dalam modul-modul independen yang memiliki tanggung jawab tunggal.

Tujuan utama dari pendekatan modular ini adalah:

1. Mencegah terjadinya regresi fungsional (functional regression) saat melakukan pembaruan kode.
2. Memudahkan proses pengujian unit secara terisolasi.
3. Mempercepat iterasi pengembangan komponen secara paralel.

Dengan mengisolasi parameter kompleks ke dalam bagian-bagian terkontrol, para insinyur dapat meminimalkan ketergantungan antar-modul dan menjaga kestabilan operasional sistem keseluruhan secara maksimal. Hal ini terbukti mampu menghemat waktu pengembangan serta memperkecil risiko kesalahan ketika sistem berkembang menjadi lebih kompleks.

Bab 2: Database Vektor & Ekstensi pgvector

Pencarian semantik berkinerja tinggi memerlukan infrastruktur penyimpanan khusus yang dioptimalkan untuk representasi vektor berdimensi tinggi. Database vektor hadir sebagai solusi untuk menyimpan, mengindeks, dan melakukan kueri cepat pada representasi matematis dari teks atau materi pembelajaran.

Di dalam ekosistem PostgreSQL, mengaktifkan ekstensi pgvector memungkinkan kita untuk menyimpan representasi embedding secara langsung dalam kolom database. pgvector mendukung pembuatan indeks menggunakan struktur khusus seperti IVFFlat (Inverted File with Flat Compression) atau HNSW (Hierarchical Navigable Small World).

Struktur indeks ini memungkinkan pencarian kemiripan vektor dilakukan dalam hitungan milidetik. Pencarian ini dihitung berdasarkan rumus matematis Jarak Cosine (Cosine Distance) untuk mengukur sudut perbedaan arah antara dua vektor dalam ruang berdimensi tinggi. Formula ini dinyatakan sebagai:

$$\text{Cosine Similarity} = (a \cdot b) / (||a|| * ||b||)$$

Dengan menggunakan pgvector, kita dapat dengan mudah menemukan potongan paragraf yang paling relevan dari ribuan dokumen hanya dengan membandingkan nilai kemiripan vektornya.

Bab 3: Menganalisis & Mengatasi Overfitting

Salah satu tantangan terbesar dalam melatih model machine learning adalah fenomena overfitting. Model yang mencapai akurasi pelatihan 99% tetapi turun menjadi 60% pada validasi menunjukkan fenomena overfitting. Ini adalah kondisi di mana model machine learning menghafal detail spesifik dan derau (noise) pada data pelatihan alih-alih mempelajari pola umum yang universal.

Ciri-ciri utama model yang mengalami overfitting meliputi:

1. Akurasi data pelatihan sangat tinggi (mendekati sempurna).
2. Akurasi data pengujian atau validasi sangat rendah dan tidak stabil.
3. Model memiliki varians yang sangat tinggi terhadap perubahan kecil pada input.

Untuk mengatasi overfitting, para praktisi machine learning disarankan menerapkan strategi berikut:

- Menerapkan regularisasi L1 (Lasso) atau L2 (Ridge) untuk memberikan penalti pada bobot yang terlalu besar.
- Menyederhanakan arsitektur jaringan saraf dengan mengurangi jumlah lapisan (layers) atau neuron.
- Memperluas serta menambah variasi dataset pelatihan melalui teknik augmentasi data atau pengumpulan sampel baru.
- Menggunakan teknik early stopping untuk menghentikan pelatihan sebelum model mulai menghafal noise.

Bab 4: Arsitektur RAG & Efisiensi Token

Retrieval-Augmented Generation (RAG) is a highly efficient technique that combines search retrieval with generative LLMs. Retrieval-Augmented Generation (RAG) adalah metode efisien yang menggabungkan pencarian dokumen dengan LLM generatif. Mengirimkan keseluruhan isi file PDF ke model bahasa besar (LLM) seperti OpenAI adalah tindakan yang sangat tidak efisien karena menghabiskan batas token konteks yang besar dan memakan biaya operasional yang sangat mahal.

Arsitektur RAG memecahkan masalah ini melalui alur kerja terstruktur:

1. Dokumen PDF dipotong menjadi paragraf kecil menggunakan metode pemotongan kalimat (semantic chunking).
2. Setiap paragraf dikonversi menjadi representasi vektor embedding (misalnya 1536 dimensi).
3. Ketika pengguna mengajukan pertanyaan, sistem mencari 3 hingga 5 paragraf teratas yang paling mirip menggunakan pgvector.
4. Hanya potongan paragraf terpilih tersebut yang dikirimkan ke OpenAI sebagai konteks pendukung untuk dijawab.

Pendekatan selektif ini secara dramatis meminimalkan konsumsi token OpenAI, meningkatkan kecepatan kueri, dan menjamin jawaban yang berlandaskan sumber data asli tanpa adanya halusinasi informasi.